Section A: Concept Application

1. Identify the string methods and their execution order needed to remove extra spaces and fix inconsistent casing in a product name.

Answer:

To clean a product name, first remove unnecessary spaces and then standardize the letter casing.

Execution order:

1. `strip()` – removes leading and trailing spaces.
2. `split()` – breaks the string into words and removes extra spaces between them.
3. `' '.join()` – joins the words back together with a single space.
4. `title()` (or `lower()`/`upper()` depending on the requirement) – standardizes the casing.

Example:

```
product = "   wIRELESS    mouse   "
cleaned_product = " ".join(product.strip().split()).title()
```

Output:

```
Wireless Mouse
```

---

2. Explain why manual CSV loading often treats numeric columns as strings and identify the function used to convert them to floats.

Answer:

When a CSV file is read manually, each value is initially read as text because CSV files store data as character strings separated by commas. Python does not automatically know whether a value represents a number, date, or text.

To convert a numeric string into a floating-point number, the **`float()`** function is used.

Example:

```
price = "49.99"
price = float(price)
```

After conversion, `price` becomes a numeric value that can be used in calculations.

---

3. Determine the best data structure for mapping product names to total sales and explain the drawback of using a list instead.

Answer:

A **dictionary** is the best data structure because it stores information as **key-value pairs**, where the product name can be the key and total sales can be the value.

Example:

```
sales = {
    "Laptop": 5000,
    "Mouse": 800,
    "Keyboard": 1200
}
```

Why not use a list?
A list requires searching through all elements to find a particular product, which is slower and less efficient for lookups.

Example:

```
sales = [("Laptop", 5000), ("Mouse", 800)]
```

Finding a product in a list may require checking multiple items one by one, whereas a dictionary provides direct access using the product name.

---

4. Compare using a manual `for` loop versus the `sum()` function, identifying a scenario where the loop is the only viable option.

Answer:

The **`sum()`** function is concise and efficient when the goal is simply to add all numeric values in a collection.

Example:

```
numbers = [10, 20, 30]
total = sum(numbers)
```

A **manual `for` loop** is more flexible because additional processing or conditions can be applied before adding values.

Example:

```
total = 0
for value in numbers:
    if value > 10:
        total += value
```

A loop is the only viable option when:

- Values need validation before summing.
- Certain values must be excluded.
- Data must be transformed during processing.
- Multiple operations are performed simultaneously.

---

5. Describe how to implement type checking or error handling to prevent a `TypeError` when comparing a potential string variable to a number.

Answer:

A `TypeError` occurs when Python tries to compare incompatible types, such as a string and an integer.

Method 1: Type Checking

```
if isinstance(value, (int, float)):
    if value > 10:
        print("Valid comparison")
```

This ensures the comparison is performed only on numeric values.

Method 2: Error Handling

```
try:
    if float(value) > 10:
        print("Valid comparison")
except (ValueError, TypeError):
    print("Invalid numeric value")
```

This approach attempts conversion and safely handles invalid data without crashing the program.

---

6. Explain the benefits of modularizing a data pipeline into separate functions and the risks of using a single large code block in analytics.

Answer:

Benefits of modularization:

- Improves code readability and organization.
- Makes debugging easier because each function has a specific task.
- Encourages code reuse.
- Simplifies testing of individual components.
- Makes maintenance and future updates easier.
- Supports teamwork by allowing different developers to work on separate modules.

Example:

```
def load_data():
    pass

def clean_data():
    pass

def analyze_data():
    pass

def generate_report():
    pass
```

Risks of using a single large code block:

- Difficult to understand and maintain.
- Errors are harder to locate and fix.
- Code becomes repetitive and less reusable.
- Small modifications may affect unrelated sections.
- Collaboration becomes more challenging.

Therefore, breaking a data pipeline into separate functions produces cleaner, more maintainable, and more reliable analytics code.